

Speed Programme Phase 4 / P4-T04 — profile-gated alloc/ GC/contention tuning

Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Authority	docs/research/speed/04-phased-implementation-plan.md — P4-T04

Table of contents

- [1. Mandate](#)
- [2. Candidates assessed](#)
- [3. B22 — internal/memory — outcome](#)
- [4. B23 — internal/cognee — outcome](#)
- [5. Captured profile artefacts](#)

1. Mandate

P4-T04 tunes **only** hot paths where Phase-0 pprof evidence proves a real bottleneck (constitution §11.4.6 — no-guessing). Both R1 audit candidates (B22, B23) were marked **UNCONFIRMED** in 01-current-state-bottleneck-audit.md. This round captured mutex/block/CPU profiles to confirm or reject each before touching code. A change without a profile proving the bottleneck is a §11.4.6 violation and was not made.

2. Candidates assessed

Candidate	R1 ref	Audit status	This round's verdict
internal/memory/memory_manager.go lock density	B22	UNCONFIRMED	Manager-level read-mux NOT confirmed hot

Candidate	R1 ref	Audit status	This round's verdict
internal/cognee/service.go ServiceCache mutex	B23	UNCONFIRMED	(1.5% of mutex delay). One real correctness defect found + fixed (double-RLock). NOT confirmed — no pathological contention. Left untouched.

3. B22 — internal/memory — outcome

The “32 lock sites” in `memory_manager.go` are spread across **six independent structs**, each with its own `sync.RWMutex` (`MemoryManager`, `InMemoryProvider`, `RedisMemoryProvider`, `MemcachedMemoryProvider`, `FilesystemMemoryProvider`, `factory`) — not one hot lock. The mutex profile (`memory-mutex-BEFORE.pprof`) under a deliberately write-mixed `RunParallel` benchmark attributes:

- **95.4%** of mutex delay to `InMemoryProvider.Store`’s write-lock — this is *inherent* `RWMutex` writer-serialization for a `map+mutex` design, NOT a pathology. `InMemoryProvider` is a dev/fallback provider; production uses `Redis/Memcached/Filesystem`. Converting it (sharding / `sync.Map`) would be a *speculative* optimization the profile does not justify — **left untouched**.
- **only 1.5%** to the `MemoryManager` read-mux — the B22 hypothesis is **NOT confirmed**.

Confirmed defect (fixed): `GetDefaultProvider()` held `mm.mu.RLock()` then re-entered `GetProvider()` which acquires the *same* `RWMutex` for reading again. Go’s `sync.RWMutex` is **not reentrant** — a writer’s `Lock()` queued between the outer and inner `RLock` deadlocks all three. Every `Store/Retrieve/Search/Delete/Clear` call funnels through `GetDefaultProvider`, so this sits on the genuine hot path. Fix: resolve the provider inline under the single existing `RLock` — removes the deadlock window AND narrows the critical section to one lock round-trip.

Evidence — `BenchmarkMemoryManager_GetDefaultProvider` (isolated path):

	ns/op	allocs/op
BEFORE	116.7	0
AFTER	67.75	0

	ns/op	allocs/op
ratio	1.72× faster	—

Block-profile focus on GetDefaultProvider: 2.63s → 2.19s cum delay.
-race clean —

TestMemoryManager_GetDefaultProviderNoReentrantDeadlock (64 readers × 2000 iter vs 8 contending writers) passes; it reproduces and guards the deadlock window.

4. B23 — internal/cognee — outcome

ServiceCache’s methods (getCachedMemory, cacheSearch, etc.) are all single-level, well-formed RLock/Lock holding the lock only for the map op. The mutex profile (cognee-mutex-BEFORE.pprof) shows the same shape as memory: the write path of the RWMutex dominates under a write-mixed benchmark — *inherent* serialization, not a pathology. No double-lock, no over-broad critical section, no production-driven concurrent hot path comparable to memory’s GetDefaultProvider. **B23 is NOT confirmed — internal/cognee was left untouched.** Profiles are committed as the negative evidence.

5. Captured profile artefacts

File	What it proves
memory-mutex-BEFORE.pprof / -AFTER.pprof	Manager read-mux only 1.5% — B22 not confirmed; Store write-path inherent and unchanged.
memory-block-BEFORE.pprof / -AFTER.pprof	GetDefaultProvider block delay 2.63s → 2.19s.
memory-cpu-BEFORE.pprof / -AFTER.pprof	CPU shape of the memory benchmarks.
cognee-mutex-BEFORE.pprof / -AFTER.pprof	No pathological contention — B23 not confirmed (control: code unchanged).
cognee-block-BEFORE.pprof / -AFTER.pprof	Same.
cognee-cpu-BEFORE.pprof	CPU shape of the cognee cache benchmark.

Capture command (per package): GOMAXPROCS=8 go test -run='^\$'
-bench=... -benchmem -benchtime=2s -mutexprofile=...
-blockprofile=... -cpuprofile=....